Engineering Decision Record

SENTINEL

AI Prompt Security Gateway

A browser-native security layer that detects and redacts sensitive data and prompt-injection attacks before they reach ChatGPT, Claude, or Gemini — in under 5 ms, local-first, with nothing stored but anonymized metadata.

Chrome Manifest V3 | Vanilla-JS engine | Trie + Aho-Corasick | React + Vite | FastAPI + Postgres | 91 automated tests | CI/CD on GitHub

Document: Build decisions, trade-offs & rationale — from scratch to deployment

Author: alirizzzv · **Repository:** github.com/alirizzzv/SENTINEL · **Live:** alirizzzv.github.io/SENTINEL

Purpose of this doc: a software-engineer–facing record of every significant decision and what it demonstrates.

1 · Purpose & problem

People paste API keys, credentials, customer PII, and internal documents into LLMs every day to "just debug this" or "clean up this paragraph." The instant they hit send, that data lives on a third party's servers — potentially used for training, potentially exposed in a breach, permanently outside their control. Existing enterprise DLP tooling is expensive, IT-gated, and *reactive* — it audits *after* the leak.

SENTINEL's thesis: interception must happen *before* the prompt is sent, must run locally so the privacy tool isn't itself a privacy risk, and must be an *education moment* (warn & let the user decide) rather than a hard block that trains people to disable it.

Who it's for	What it does
The accidental insider — an employee using AI tools who doesn't realize they're leaking data. Secondary: IT admins wanting team-level visibility (opt-in).	Intercepts every send, scans in <5 ms, scores risk 0–100, shows what was found ranked by severity, and offers Redact / Send Anyway / Cancel.

2 · System at a glance

Three cooperating parts, all local by default:

Layer	Responsibility	Tech
Detection engine	Turn a string into a risk verdict + redacted text	Vanilla JS (ES modules)
Extension	Intercept the send on each LLM site; show the modal; store metadata	Manifest V3 content script + service worker + IndexedDB
Surfaces	Popup quick-stats; full analytics dashboard	React + Vite + Chart.js
Backend (optional)	Org-level analytics from anonymized metadata	FastAPI + SQLAlchemy + Postgres/SQLite

Keystone decision: the engine is **one source of truth** — the same ES-module code is unit-tested by Vitest *and* bundled (esbuild) into the content script, so what is tested is exactly what ships. This single choice removes an entire class of "works in tests, breaks in prod" drift.

3 · Decision log

Each entry: the context, the options weighed, the decision, its pros and trade-offs, and what it enables / what it signals as engineering judgment.

D1 · Ship as a browser extension (not a web app or desktop proxy)

Context: the tool must act at the exact moment a prompt is sent, with zero change to user habits.

Decision: a Chrome/Chromium extension that injects into LLM pages.

PROS

- Intercepts in-context — no copy/paste to a separate site (which no one does consistently).
- Works across every LLM with one install.
- Runs on the user's machine → privacy by construction.

TRADE-OFFS

- Coupled to each site's DOM (mitigated by the adapter pattern, D11).
- Web Store review for public distribution.

Demonstrates: product judgment — choosing the form factor that actually changes behavior, not the one that's easiest to build.

D2 · Manifest V3

Context: MV2 is deprecated; MV3 is the current, required extension platform.

Decision: MV3 with a service worker, content scripts, and a strict minimal permission set.

PROS

- Future-proof; required by the Web Store.
- Enforced CSP (no inline/eval) → smaller attack surface.
- Event-driven service worker = low idle cost.

TRADE-OFFS

- Service workers are ephemeral → state must live in IndexedDB, not memory.
- Background can't run the engine synchronously for the content script (drove D10).

Demonstrates: working within real platform constraints rather than against them.

D3 · Detection engine in plain JavaScript (ES modules), framework-free

Context: the engine must load instantly, run inside a content script, and be portable to Node tests.

Decision: vanilla ES modules with no runtime dependencies.

PROS

- Zero dependencies → tiny, instant, auditable.
- Same code runs in Vitest, the content script, and the demo.
- No build tooling needed to reason about correctness.

TRADE-OFFS

- No static types (mitigated with JSDoc + an exhaustive test suite).

Demonstrates: picking the right weight of tool — frameworks where they pay (the dashboard), nothing where they'd only add cost (a hot-path library).

D4 · Aho-Corasick built from scratch (not sequential regex, not a library)

Context: scan a prompt against 20+ patterns fast enough to be invisible, and safely on hostile input.

Decision: a hand-built Trie + Aho-Corasick automaton (BFS failure & output links) — a DFA that finds all matches in one linear pass.

PROS

- `O(n + m + z)` regardless of pattern count — vs `O(k·n)` for k sequential regexes.
- DFA = no backtracking → immune to catastrophic ReDoS on adversarial input.
- Demonstrates the algorithm rather than hiding it behind a dependency.

TRADE-OFFS

- More code to write and test than calling a library.
- Anchorless PII (email/phone/card) has no literal prefix → handled by a small always-on regex set (D5).

Demonstrates: core DSA depth (tries, BFS, automata, complexity analysis) and security awareness — the exact thing a DLP / antivirus / IDS engine uses.

D5 · Two-stage detection: cheap candidates → strict validation

Context: Aho-Corasick finds candidates ("AKIA"), but a fragment isn't a confirmed key.

Decision: Aho-Corasick flags candidates; a strict regex confirms structure; an optional callback (e.g. the **Luhn checksum** on card numbers) cuts false positives.

PROS

- Speed (fast gate) + accuracy (precise validation) — the grep / Elasticsearch pattern.
- Luhn rejects random 16-digit IDs that merely look like cards.

TRADE-OFFS

- Two representations per pattern (anchor + regex) to keep in sync.

Demonstrates: precision/recall thinking and pragmatic pipeline design.

D6 · Sub-match suppression (merge-intervals containment)

Context: a credit card's first 12 digits also match the Aadhaar pattern → a phantom second threat. (Found by the test suite, not in prod.)

Decision: drop any match fully contained inside a longer, at-least-as-severe match of a different pattern.

PROS

- Honest threat lists; no double counting.
- Guarded by a regression test so it can't silently return.

TRADE-OFFS

- A small extra pass over matches (negligible).

Demonstrates: debugging discipline — a real bug, root-caused and locked down with a test.

D7 · Risk scoring with a hand-built max-heap

Context: show the user the scariest finding first; combine multiple findings into one score.

Decision: push threats into a binary max-heap, pop in descending severity; composite = top score + 5×(extra threats), capped at 100; bands SAFE/CAUTION/HIGH.

PROS

- Right structure for "ordered by priority" — `O(k log k)`.
- Explainable, deterministic scoring.

TRADE-OFFS

- k is tiny, so a sort would also do — the heap is a deliberate DSA choice.

Demonstrates: matching data structure to problem and reasoning about it out loud.

D8 · Three-layer injection detection with a confidence threshold

Context: "ignore previous instructions" is an attack; "summarize the previous email" is not. One signal is never enough.

Decision: (1) known phrases via a dedicated Aho-Corasick, (2) structural heuristics, (3) a confidence score gated by a user sensitivity setting (LOW/MED/HIGH).

PROS

- Drives false positives down — the failure mode that makes users disable a tool.
- Tunable; never hard-blocks; explains its signals.

TRADE-OFFS

- Heuristic, not semantic — misses novel/obfuscated phrasings (future: ML layer).

Demonstrates: ML-adjacent precision/recall judgment and humane UX in security.

D9 · Redaction via merge-intervals

Context: detected spans can overlap (a Bearer token whose value is a JWT); naive replacement corrupts text.

Decision: sort spans, merge overlaps (keep the higher-severity placeholder), then one left-to-right slice pass.

PROS

- `O(n log n)`, correct on overlaps, single rebuild pass.

TRADE-OFFS

- Placeholder choice on collisions is a heuristic (severity-first).

Demonstrates: the classic merge-intervals problem applied to a real feature.

D10 · Synchronous in-page detection + esbuild single-source bundle

Context: `preventDefault()` must be called synchronously to stop a send — so detection must be synchronous too, but the engine "lives" as ES modules.

Decision: esbuild bundles the tested engine into a global the content script runs in-page; only metadata storage is async.

PROS

- Reliable interception (no async race on the hot path).
- One source of truth — tested code == shipped code.

TRADE-OFFS

- A build step (resolved with a tiny, fast esbuild script).

Demonstrates: understanding the event loop, browser event model, and build pipelines.

D11 · Adapter pattern registry (open/closed principle)

Context: each LLM site finds its input/send differently and changes its DOM over time.

Decision: a hash map keyed by hostname; each adapter is a config of fallback selectors. `O(1)` lookup; new LLM ≈ 5 lines.

PROS

- Decouples *what* we detect from *where* — engine knows nothing about sites.
- Zero overhead on unsupported sites; resilient to layout changes.

TRADE-OFFS

- Selectors still need occasional maintenance on major redesigns.

Demonstrates: design patterns & SOLID applied where they genuinely reduce churn.

D12 · Local storage = IndexedDB circular buffer (not localStorage)

Context: store usage history without blocking the UI or growing unbounded.

Decision: IndexedDB, bounded at 500 events with FIFO eviction; **metadata only** — never prompt text or detected values.

PROS

- Async (non-blocking), 50 MB+, indexable range queries.
- Circular buffer = `O(1)` amortized, bounded memory.

TRADE-OFFS

- More API ceremony than localStorage (worth it).

Demonstrates: storage trade-offs (sync vs async), and OS-style circular-buffer thinking.

D13 · Local-first privacy + least-privilege permissions

Context: a security tool that exfiltrates data is itself the risk ("the privacy paradox").

Decision: no network in the scan path; permissions limited to `storage` + the three LLM hosts; enterprise sync is strictly opt-in to the org's own server.

PROS

- Verifiable in DevTools (zero outbound requests).
- Trust through least privilege and open source.

TRADE-OFFS

- No central analytics unless a user/org explicitly opts in.

Demonstrates: threat-model thinking and a principled privacy stance.

D14 · Dashboard in React + Vite + Chart.js (framework *here*, not in the engine)

Context: the dashboard is a stateful SPA (tabs, filters, charts) — different problem than the hot-path engine.

Decision: React + Vite, built to static files the extension loads; Chart.js for charts; a data layer that falls back to seeded sample data outside the extension (so it's demoable anywhere).

PROS

- Component model fits complex UI state; fast Vite builds.
- Same dashboard runs on GitHub Pages with mock data.

TRADE-OFFS

- Larger bundle than vanilla — acceptable for a non-hot-path SPA.

Demonstrates: right-tool-for-the-job judgment and modern frontend tooling.

D15 · Warm light visual identity (brand differentiation)

Context: needed a distinct, premium identity (a sibling project was already dark).

Decision: a warm light theme with violet accents, glassmorphism, pastel-aurora backgrounds, and subtle 3D tilt; dark theme kept as a toggle.

PROS

- Premium, modern, recognizable; strong portfolio impression.

TRADE-OFFS

- Light themes need careful contrast tuning (handled with tokens).

Demonstrates: product/visual polish and CSS architecture (theming via custom properties).

D16 · Backend: FastAPI + SQLAlchemy + Pydantic; SQLite by default, Postgres for prod

Context: optional enterprise analytics needs a typed, documented API that runs with zero setup yet scales.

Decision: FastAPI (async, auto OpenAPI), Pydantic validation, SQLAlchemy 2.0; `DATABASE_URL` selects SQLite (dev) or Postgres (prod) with no code change.

PROS

- Zero-setup demo (SQLite) → real DB by env var.
- Free Swagger docs; request validation at the edge.

TRADE-OFFS

- A portable column type needed so one schema serves both DBs.

Demonstrates: backend/API design, ORMs, and dev/prod parity.

D17 · Backend security: hashed keys, org-scoping, composite indexes

Context: multi-tenant analytics must isolate orgs and never store raw secrets.

Decision: API keys stored as SHA-256 hashes only; every query filtered by `org_id`; composite indexes `(org_id, timestamp)`, `(org_id, risk_level)`, `(org_id, llm)`.

PROS

- Tenant isolation; no plaintext credentials at rest.
- Index-backed reads — one org never scans another's rows.

TRADE-OFFS

- Indexes cost write throughput (fine for read-heavy analytics).

Demonstrates: multi-tenant data modeling, indexing, and security hygiene.

D18 · Test-first, multi-layer testing

Context: a security tool's credibility is its test suite; false positives are the real enemy.

Decision: Vitest unit tests per structure; a positive/negative **corpus**; an **adversarial** suite (ReDoS, obfuscation, false-positive sweep); IndexedDB tests via `fake-indexeddb`; a perf **bench**; pytest for the API. **91 tests total.**

PROS

- Regressions caught instantly (it already caught a real UUID→Aadhaar bug).
- Performance claims are measured, not asserted.

TRADE-OFFS

- Upfront effort — repaid many times over.

Demonstrates: testing maturity, adversarial thinking, and benchmarking.

D19 · Deployment: GitHub Actions CI/CD + Pages; packaged extension; containerized backend

Context: the project must build, test, and publish itself reproducibly.

Decision: a CI workflow (engine + backend tests on every push) and a Pages workflow that builds and publishes the live demo + dashboard; `npm run package:ext` produces the store-ready zip; `Dockerfile + render.yaml` make the backend one-click deployable.

PROS

- Every push is verified and the demo auto-redeploys.
- Reproducible builds; clear release artifact.

TRADE-OFFS

- Chrome Web Store publishing still needs a developer account + review.

Demonstrates: CI/CD, containerization, and a real release process.

4 · Language choices

Where	Language	Why this, not the alternative
Detection engine, extension	**JavaScript** (ES modules)	It is the only language that runs natively inside a browser content script. Plain JS keeps the hot path dependency-free and lets the same code be unit-tested in Node. TypeScript was weighed; JSDoc + a deep test suite gave most of the safety without a build step on the engine.
Dashboard	**JavaScript** + JSX (React)	Shares the ecosystem; React's component model suits the SPA's state.
Backend	**Python** (FastAPI)	Fast to build a typed, documented, async API; Pydantic validation; ubiquitous for data work. Node/Express was viable but FastAPI's auto-OpenAPI and Pydantic won for an analytics service.
Tooling/CI	**YAML** + shell + Node scripts	Standard GitHub Actions; small Node scripts for engine bundling, site assembly, and PDF/screenshots.

5 · Data-structures & complexity map

Feature	Structure / algorithm	Complexity
Multi-pattern scan	Aho-Corasick (Trie + BFS failure links → DFA)	`O(n + m + z)`
Threat prioritization	Binary max-heap	`O(k log k)`
Per-site adapter lookup	Hash map	`O(1)`
Local event storage	Circular buffer (FIFO)	`O(1)` amortized
Redaction / overlap merge	Merge intervals	`O(n log n)`
Candidate → validation	Two-stage filtering	—
Backend range queries	B-tree (composite indexes)	`O(log n)`

6 · Measured performance

Node 25, `npm run bench`. Linear scaling, no catastrophic backtracking.

Prompt	Size	p50	p99
typical prompt	~50 B	0.002 ms	0.01 ms
realistic mixed	~1 KB	0.023 ms	0.045 ms
large document	~10 KB	0.26 ms	0.38 ms
adversarial near-miss	~80 KB	2.4 ms	2.6 ms

7 · Limitations & future work

- **Injection is heuristic, not semantic** — Unicode-homoglyph and indirect (data-borne) injection aren't covered; a future ML layer would catch novel phrasings.
- **Selector fragility** — major LLM redesigns can require an adapter update; planned mitigation: Playwright tests run daily against live sites.
- **Pattern updates ship with the extension** — a CDN-hosted remote pattern registry (cached in IndexedDB, bundled fallback) would enable minute-level updates.
- **Scale** — detection is free (a million parallel in-browser engines); the backend would shard `sync_events` by `org_id` and use TimescaleDB for trend data.

8 · What this demonstrates for a software-engineer role

Competency	Evidence in SENTINEL
Algorithms & DSA	Aho-Corasick + Trie from scratch, max-heap, merge-intervals, circular buffer — with complexity reasoning.
Systems thinking	MV3 service-worker/content-script split, sync-vs-async on the hot path, single-source bundling.
API & data modeling	FastAPI + Pydantic, multi-tenant org-scoping, composite indexes, dev/prod DB parity.
Frontend	React + Vite SPA, Chart.js, CSS theming, accessible interaction, 3D/visual polish.
Security mindset	ReDoS-safe by design, least-privilege, hashed credentials, threat modeling, false-positive discipline.
Quality & testing	91 tests across unit/corpus/adversarial/persistence + perf benchmark; a real bug found and regression-locked.
Delivery	CI/CD on GitHub Actions, live Pages deploy, packaged extension, containerized backend, thorough docs.
Judgment	Framework where it pays, none where it doesn't; warn-don't-block UX; privacy by construction.

9 · Anticipated interview questions & answers

The questions a skeptical interviewer is most likely to ask — and crisp answers grounded in the build.

Q Explain Aho-Corasick.

A Insert all patterns into a trie, then add *failure links* via BFS — each node points to the longest proper suffix of its path that's also a prefix in the trie. Output links merge so each node reports every pattern that ends at it or any failure-reachable node. Scanning walks the text once; on a mismatch you follow the failure link instead of restarting. It's a DFA, so total time is `O(n + m + z)` — text + patterns + matches — independent of how many patterns there are.

Q Why not just use regex for everything?

A Three reasons. Performance: 20 sequential regexes are `O(k·n)`; Aho-Corasick is `O(n)` regardless of pattern count. Safety: regex backtracking can blow up (catastrophic ReDoS) on adversarial input and hang the tab — a DFA never backtracks. Composition: a trie shares prefix work across patterns. I still use regex as the *second* stage to validate the exact structure of candidates the automaton flags.

Q Why a browser extension instead of a website or API?

A Behavior. A website needs the user to copy → paste → check → paste back; nobody does that consistently. An extension intercepts at the exact moment of sending with zero habit change, works across every LLM with one install, and runs on the user's machine so it's private by construction.

Q How do you know detection actually works? How is it tested?

A 91 automated tests: unit tests for the trie/automaton/heap/redactor; a *corpus* of known-positive prompts (must flag) and known-negative prompts (must stay safe) as a false-positive guard; an *adversarial* suite (ReDoS, obfuscation, dev-text false-positive sweep); IndexedDB tests via fake-indexeddb; and a performance benchmark. CI runs all of it on every push. The corpus already caught a real bug — a UUID segment matching the Aadhaar pattern — which I fixed and regression-locked.

Q What was the hardest engineering problem?

A Reliable interception across three different editors while keeping detection synchronous. `preventDefault()` must run synchronously to stop a send, so the engine can't be an async call to the background worker — I bundle it into the content script with esbuild so it runs in-page. Then redaction has to actually "stick" in rich editors: I use the native value setter for React textareas and `execCommand('insertText')` for ProseMirror/Quill contenteditable, which route through the editor's own input pipeline.

Q You claim you never store prompt content — prove it.

A The extension requests only `storage` plus the three LLM hosts — no broad host permissions — so the browser itself prevents exfiltration. There are zero network requests on the scan path (verifiable in DevTools → Network). Only anonymized metadata (timestamp, LLM, risk level/score, threat *categories*, decision, timing) goes into a local 500-event ring buffer. The source is open and auditable.

Q How do you keep false positives down?

A A false positive that blocks a legitimate prompt trains users to disable the tool — worse than a missed detection. So: a Luhn check on card-shaped numbers, lookarounds so IDs/UUIDs don't masquerade as Aadhaar, sub-match suppression for contained overlaps, and — for injection — a confidence score that requires multiple signals before flagging (one weak signal like "act as a…" never fires). And we never hard-block; the user always decides.

Q What happens when two detected spans overlap?

A Classic merge-intervals: sort spans by start, sweep once merging overlaps, keep the higher-severity placeholder on a collision, then rebuild the string in a single slice pass — `O(n log n)`. For scoring, a span fully contained in a longer, equally/more severe span of a different pattern is dropped so we don't double-count.

Q How would you scale this to a million users?

A Detection scales for free — it's a million parallel in-browser engines, zero server load. Only the optional enterprise backend scales: horizontal FastAPI behind a load balancer, shard `sync_events` by `org_id`, a read replica for analytics, and TimescaleDB for the time-series trend data. Every query is already org-scoped and index-backed.

Q Patterns ship with the extension — how do you update them instantly?

A Today a pattern change needs a new release (Web Store review = days). The fix is a CDN-hosted JSON pattern registry the extension fetches on startup and caches in IndexedDB, with the bundled dictionary as an offline fallback — giving minute-level updates without a release.

Q What's your threat model — and how do you prevent the extension itself being the risk?

A Three actors: the accidental insider (primary), the attacker crafting injections, and the LLM provider using data for training. We're a guardrail, not a wall — a determined insider can disable any extension. The extension protects itself by design: no server to exfiltrate to, least-privilege permissions, MV3's CSP (no eval), and open source for audit.

Q Why Python for the backend but JavaScript for the engine?

A JS is the only language that runs inside a browser content script, and plain JS keeps the hot path dependency-free while still being testable in Node. The backend is an optional analytics service where FastAPI's async model, automatic OpenAPI docs, and Pydantic validation make it the fastest path to a correct, documented API — so I used the best tool per layer rather than forcing one language everywhere.

Q What would you improve with more time?

A An ML layer for semantic/obfuscated injection detection, Playwright tests running daily against the live LLM sites to catch selector drift, the remote pattern registry for instant updates, and Firefox support. These are documented in the limitations section, not hidden.

In one line: SENTINEL is a from-scratch, test-driven, security-focused full-stack system that shows algorithmic depth, platform fluency, and the engineering judgment to choose the simplest tool that actually solves the problem — built, measured, documented, and deployed end-to-end.